HATHOR-CORE · MASTER-BOOK

Nano-Contracts — The Smart-Contract Runtime

Hathor's on-ledger programs — Blueprints written in Python, called by transactions, executed with metered resources against verifiable trie-stored state.

SUBJECT	hathor-core · Part II · the node, end to end
CHAPTER	39 · Part II · The Node
BRANCH	study/hathor-core-studies
EDITION	2026 · v1

Nano-contracts · Blueprints · @public/@view · Runner · Metered execution · Gas/fuel · Patricia/Merkle trie · Contract state · NCActions · On-chain blueprints

Table of Contents

Chapter 39 — Nano-Contracts: The Smart-Contract Runtime

What you'll learn in this chapter

- What a smart contract is in general, and how Hathor's **nano-contract** differs from Ethereum's model — Python Blueprints run on the node's own interpreter, not a bespoke virtual machine.
- The anatomy of a **Blueprint**: the base class, the `@public` / `@view` / `@fallback` decorators, and how a plain type annotation becomes a piece of persistent state.
- How a transaction calls a contract: the **nano-header** it carries, the **NCActions** that move tokens in and out, and the **Context** the method receives.
- The **Runner** — the engine that dispatches a call to a Blueprint method — and why execution must be **metered** (the halting problem, resource exhaustion, the "never block" rule from Chapter 2).
- Where contract state lives: a **Patricia/Merkle trie**, what that data structure is, and why it gives every contract a verifiable state root.
- How execution ties into **consensus**: contracts run when a block confirms them, in a deterministic order; a contract that fails is **voided** (Chapter 32), and the whole subsystem is gated behind the `NANO_CONTRACTS` feature (Chapter 38).

This is the largest single subsystem in `hathor-core` — roughly fifteen thousand lines across eighty-odd files. It is also the one that asks the most of you conceptually, because it layers a second world on top of the ledger you have spent the whole book learning. Underneath, there is still the DAG of vertices, the UTXO accounting, the consensus that picks a canonical history. On top of it now sits a small programming language runtime: programs, state, function calls, resource limits.

We split the chapter into two parts. **Part A — the runtime** answers what runs, and how: what a Blueprint is, how a transaction invokes one of its methods, and how the Runner executes that method safely. **Part B — the state** answers where it lives, and why it is verifiable: the trie that stores every contract's data, the typed fields that serialize into it, and the way a failed execution is folded back into consensus. Read Part A first; Part B will then make sense as "and here is where all those state changes actually land."

One framing sentence to hold onto throughout: **a nano-contract is a vertex that owns a balance and runs code.** Everything else is detail on top of that.

39.A — The Runtime

39.A.1 What a smart contract is

Before any Hathor code, the general idea.

A **smart contract**[1] is a program stored on a ledger whose execution is performed and agreed upon by every node in the network, not by one trusted server. The phrase is older than blockchains, but blockchains are what made it practical, because a blockchain already gives you the two things a contract needs and a normal program cannot assume:

1. **Shared, tamper-evident state.** Every node holds the same copy of the data, and nobody can secretly alter it.
2. **Agreement on what happened.** Consensus (Chapter 32) means that when the contract's code runs, every node computes the same result and records the same state change. There is no "it worked on my machine."

The canonical mental model is a vending machine that holds money. Anyone can put coins in (call the contract). The machine's own rules — not a shopkeeper — decide what comes out. The rules are public and the same for everyone, and the money it holds is real. A token-swap, an escrow that releases funds when two of three people approve, a betting pool that pays winners — these are all "money plus rules," which is exactly what a contract is.

The hard part is that you are now running arbitrary user-supplied code on every node in the network, and you must make it:

- **Deterministic**[2] — every node must compute byte-identical results, or consensus breaks. No wall-clock time, no real randomness, no reading from the network.
- **Bounded** — a contract must not be able to loop forever or eat all the memory, or it could freeze every node at once (a denial-of-service against the whole network).
- **Isolated** — a contract must not be able to open files, make network calls, or reach into the node's internals.
- **Verifiable** — any node must be able to prove that the resulting state is the correct consequence of the calls, ideally with a single hash.

Every design decision in this chapter exists to satisfy one of those four constraints. Keep them in mind; they are the why behind almost everything.

How Hathor differs from Ethereum

If you have heard of smart contracts, you have heard of Ethereum, and the comparison is worth drawing because Hathor made a deliberately different choice.

Ethereum contracts are written in a language like Solidity, **compiled to bytecode**[3] for a purpose-built virtual machine — the **EVM**[4] — and that bytecode is what lives on-chain. Every node ships an EVM interpreter, executes the bytecode opcode by opcode, and charges **gas**[5] for each opcode so that execution is bounded. The EVM is a whole separate machine with its own instruction set, its own memory model, and its own notion of "an account."

Hathor's nano-contracts take a different road. A contract template — Hathor calls it a **Blueprint**[6] — is written in **a restricted subset of Python**, and it runs on the node's own Python interpreter, inside a sandbox, rather than on a separate VM. There is no Solidity, no bytecode compiler you have to learn, no second instruction set. You write a Python class; the node runs it. The class file is `hathor/nanocontracts/blueprint.py`; we will read it in a moment.

This buys ergonomics — Python developers can write contracts in a language they already know — at the cost of a harder sandboxing problem: Python was never designed to run untrusted code safely, so Hathor has to take things away from the language (disable `open`, `eval`, `import`, restrict the builtins) rather than start from a locked-down machine. The trade is "familiar language, careful jail" versus Ethereum's "purpose-built machine, unfamiliar language." We will see both the jail (§39.A.6) and the metering (§39.A.7) that together stand in for the EVM's guarantees.

> **Recap — vertex, and where contracts sit (full treatment Ch. 8 & 25).** A vertex is any node in Hathor's ledger DAG — a `Block` or a `Transaction`. In Chapter 25 you met the vertex class hierarchy, including a subclass called `OnChainBlueprint` that we deferred to here. A nano-contract is **not** a fourth vertex type; it is an ordinary `Transaction` that carries an extra header instructing the node to run contract code. The Blueprint source code itself, when stored on the ledger, rides on an `OnChainBlueprint` vertex (§39.B.5).

39.A.2 Localization

The whole subsystem lives under one package, `hathor/nanocontracts/`. Here is the map, grouped by role; the rest of the chapter walks it roughly top to bottom.

```
hathor/nanocontracts/                    ◀ YOU ARE HERE
│
├── ── WHAT A CONTRACT IS ──────────────────────────────────────
│   blueprint.py            ← Blueprint base class + metaclass (fields)
│   types.py                ← @public/@view/@fallback, NCActions, ContractId…
│   method.py               ← captures+serializes a method's signature
│   context.py              ← Context object passed to a @public method
│   on_chain_blueprint.py   ← OnChainBlueprint vertex: code stored on-ledger
│   catalog.py              ← resolves a blueprint id → blueprint class
│   blueprints/             ← built-in blueprint registry (empty by default)
│
├── ── HOW A CONTRACT RUNS ─────────────────────────────────────
│   runner/runner.py        ← THE Runner: dispatch a call to a method (1467 LOC)
│   runner/call_info.py     ← the call stack / call trace
│   metered_exec.py         ← MeteredExecutor: fuel + memory bounds
│   custom_builtins.py      ← the sandbox: which builtins a contract may use
│   blueprint_env.py        ← the "syscall" surface a contract sees
│   contract_accessor.py    ← handle for calling ANOTHER contract
│   proxy_accessor.py       ← handle for a delegatecall-style proxy
│   balance_rules.py        ← how deposits/withdrawals move balances
│   rng.py                  ← deterministic pseudo-random generator
│
├── ── WHERE STATE LIVES ───────────────────────────────────────
│   storage/patricia_trie.py    ← the Merkle/Patricia trie (the state store)
│   storage/contract_storage.py ← NCContractStorage: one contract's state
│   storage/block_storage.py    ← NCBlockStorage: all contracts in a block
│   storage/changes_tracker.py  ← stages writes so a failed call rolls back
│   fields/                      ← typed contract attributes (descriptors)
│   nc_types/                    ← serialization codecs for those types
│
└── ── EXECUTION ⇄ CONSENSUS ───────────────────────────────────
    execution/block_executor.py           ← run all NC txs in a block (pure)
    execution/consensus_block_executor.py ← apply the effects, void failures
    sorter/random_sorter.py               ← deterministic call ordering
```

The on-tx instruction that triggers a contract call does not live here — it lives with the vertex model, at `hathor/transaction/headers/nano_header.py`, because it is physically part of a transaction's bytes. We meet it in §39.A.4.

> **Context.** Nano-contracts are a higher service built on the base ledger, not part of it. The node can run with the feature switched off entirely (`ENABLE_NANO_CONTRACTS = False`), and most of mainnet's early life did. When it is on, this package is wired into three other subsystems you already know: **verification** (Chapter 31) checks that a nano-header is well-formed; **consensus** (Chapter 32) is what actually runs the contracts, when a block confirms them; and **feature activation** (Chapter 38) gates the whole thing behind the `NANO_CONTRACTS` feature so the network can switch it on by miner vote.

39.A.3 What a Blueprint is

A **Blueprint** is the template — the class — and a **contract** is a live instance of that template with its own state and balance. One Blueprint ("a betting pool") can back thousands of independent contracts (one pool per event). This is exactly the class-versus-object distinction from Chapter 1, lifted onto the ledger: the Blueprint is the class, each contract is an object, and the object's attributes are stored not in RAM but in the trie.

Let us build the intuition with a generic example before reading Hathor's real base class. Here is the shape a contract author writes — a counter that anyone can bump:

```
class Counter(Blueprint):
    count: int                      # a state attribute — persisted

    @public
    def initialize(self, ctx: Context) -> None:
        self.count = 0              # set the initial state

    @public
    def increment(self, ctx: Context) -> None:
        self.count += 1             # mutate state

    @view
    def get_count(self) -> int:
        return self.count           # read state, never mutate
```

Four things are happening here, and each is enforced by the framework, not by convention:

- `count: int` is a **state attribute**. It looks like an ordinary class annotation, but the Blueprint machinery turns it into a piece of trie-backed storage. `self.count` does not read from memory; it reads from (and writes to) the contract's slice of the state trie.
- `initialize` is the **constructor-equivalent**. Every Blueprint must define exactly one method called `initialize`, and it must be `@public`. It runs once, when the contract is created.
- `increment` is a **public method** — callable by a transaction from the outside, may change state.
- `get_count` is a **view method** — read-only, callable cheaply (e.g. from an API) without producing a transaction, and forbidden from mutating state.

Now the real base class. It is shorter than you might expect, because the cleverness lives in its **metaclass**[7].

```python
# hathor/nanocontracts/blueprint.py:126
class Blueprint(metaclass=_BlueprintBase):
    __slots__ = ('__env',)

    def __init__(self, env: BlueprintEnvironment) -> None:
        self.__env = env

    @final
    @property
    def syscall(self) -> BlueprintEnvironment:   # blueprint.py:142
        return self.__env

    @final
    @property
    def log(self) -> NCLogger:                   # blueprint.py:148
        return self.syscall.__log__
```

A Blueprint instance holds exactly one thing: `__env`, a `BlueprintEnvironment` (§39.A.6). That environment is the contract's entire connection to the outside world — its storage, its logger, its ability to call other contracts. Note `__slots__ = ('__env',)`: the instance is forbidden from storing any other attributes directly in memory. So where does `self.count` go? Into the trie — and that redirection is set up by the metaclass.

The metaclass: turning annotations into storage

A metaclass is a class whose instances are classes. When Python executes `class Counter(Blueprint): ...`, the metaclass `_BlueprintBase.__new__` runs and gets a chance to rewrite the class before it exists. Hathor uses that hook to do three jobs:

```python
# hathor/nanocontracts/blueprint.py:56-73  (abridged)
cls._validate_initialize_method(attrs)   # there MUST be an @public initialize()
cls._validate_fallback_method(attrs)
nc_fields = attrs.get('__annotations__', {})           # e.g. {'count': int}
# ... reject forbidden / underscore-prefixed field names ...
attrs[NC_FIELDS_ATTR] = nc_fields        # remember the declared fields
attrs['__slots__'] = tuple()             # forbid plain instance attributes
```

Then, for each annotated field, it replaces the annotation with a **descriptor**[8] — a `Field` object — that knows how to read and write that attribute through the trie:

```python
# hathor/nanocontracts/blueprint.py:79-92  (abridged)
for field_name, field_type in attrs[NC_FIELDS_ATTR].items():
    field = make_field_for_type(field_name, field_type)   # build a Field descriptor
    setattr(new_class, field_name, field)                  # install it on the class
```

After this runs, `Counter.count` is not an `int` slot — it is a `Field[int]` descriptor. When contract code does `self.count += 1`, Python routes the read and the write through that descriptor's `__get__`/`__set__`, which serialize and persist the value (Part B, §39.B.4). The contract author never sees any of this; they wrote `self.count` and got persistence for free. We pick up the descriptor mechanics in Part B; for now the takeaway is: **a type annotation on a Blueprint is a declaration of persistent state.**

Two structural rules the metaclass enforces, with their reasons:

- **`initialize` is mandatory and must be `@public`** (`blueprint.py:104`). A contract with no constructor could never set up its invariants; the rule guarantees there is always one well-defined entry point that runs first.
- **Field names cannot start with `_` and cannot be `syscall` or `log`** (`blueprint.py:27`, `:65`). The underscore rule keeps the contract's persisted state separate from internal plumbing; `syscall` and `log` are reserved because they are the names of the two properties through which the contract reaches the system.

39.A.4 How a transaction calls a contract

A Blueprint is inert. Something has to call one of its methods, and in Hathor that something is a transaction. The instruction to call a method is not a separate vertex type — it is an extra **header**[9] bolted onto an ordinary `Transaction`. That header is the `NanoHeader`, at `hathor/transaction/headers/nano_header.py`.

A header is a self-describing block of bytes appended to the transaction's serialized form (you met the idea in passing in Chapter 25; fee headers work the same way). The nano-header carries everything the node needs to perform one contract call:

```python
# hathor/transaction/headers/nano_header.py:100-121  (fields, abridged)
@dataclass(slots=True, kw_only=True)
class NanoHeader(VertexBaseHeader):
    nc_seqnum: int          # caller's sequence number (replay protection)
    nc_id: VertexId         # WHICH contract (or which blueprint, if creating)
    nc_method: str          # WHICH method to call, e.g. "increment"
    nc_args_bytes: bytes    # the method's arguments, serialized
    nc_actions: list[NanoHeaderAction]   # token deposits / withdrawals
    nc_address: bytes       # WHO is calling (the caller's address)
    nc_script: bytes        # the caller's signature(s) over the tx
```

Read the fields as a sentence: "caller `nc_address`, with sequence number `nc_seqnum`, calls method `nc_method` (arguments `nc_args_bytes`) on `nc_id`, attaching these token movements `nc_actions`, and proves it with signature `nc_script`." That is one contract call, fully specified.

A subtlety worth pausing on: `nc_id` means two different things depending on whether the call is creating a new contract or calling an existing one. When the method is `initialize`, the transaction is creating a contract, and `nc_id` holds the **blueprint id** (which template to instantiate). Otherwise `nc_id` holds the **contract id** (which existing instance to call). The header has a one-line helper that captures this:

```python
# hathor/transaction/headers/nano_header.py:242
def is_creating_a_new_contract(self) -> bool:
    return self.nc_method == NC_INITIALIZE_METHOD   # i.e. == "initialize"
```

And the contract's own id, when it is being created, is the creating transaction's hash (`get_contract_id`, `:247`). So a contract's identity is the id of the transaction that gave it life — the same "the vertex is its hash" principle the whole ledger runs on.

NCActions — moving tokens into and out of a contract

A counter holds no money, but most contracts do, and the question of how tokens get in and out is central. Hathor's answer is the **NCAction**. A contract has its own balance, separate from any address's balance; to move tokens across that boundary, the calling transaction declares **actions**. There are four kinds (`hathor/nanocontracts/types.py:359-491`):

Action	Direction	Meaning
`NCDepositAction`	into the contract	the tx pays tokens to the contract
`NCWithdrawalAction`	out of the contract	the contract pays tokens to the tx
`NCGrantAuthorityAction`	into the contract	give the contract mint/melt authority
`NCAcquireAuthorityAction`	out of the contract	take authority out of the contract

Each token-action names a token and an amount, and is a frozen dataclass:

```python
# hathor/nanocontracts/types.py:463-469
@dataclass(slots=True, frozen=True, kw_only=True)
class NCDepositAction(BaseTokenAction):   # token_uid + amount
    """Deposit tokens into the contract."""

@dataclass(slots=True, frozen=True, kw_only=True)
class NCWithdrawalAction(BaseTokenAction):
    """Withdraw tokens from the contract."""
```

This is where the contract world meets the UTXO world from Chapter 7. A deposit is funded by the transaction's inputs; a withdrawal becomes one of the transaction's outputs. The node enforces conservation across the boundary: the tokens a contract claims to receive must really be paid in by the tx, and the tokens it pays out must really appear as outputs. In the header the action stores a `token_index` (`nano_header.py:50`) — a pointer into the transaction's token list — rather than the token id directly, to keep the bytes small; `to_nc_action` (`:54`) resolves it back to a real `TokenUid` at execution time.

A contract author opts in to which actions a method will accept, on the decorator:

```python
@public(allow_deposit=True, allow_withdrawal=True)
def swap(self, ctx: Context) -> None:
    ...
```

A method that does not list `allow_deposit` will reject any transaction that tries to deposit into it. This is a safety default: a method receives no tokens unless it explicitly says it can.

The Context

When a `@public` method runs, it receives a `Context` as its first argument after `self`. The Context is the call's "who, what, when" — an immutable snapshot the contract can trust:

```
# hathor/nanocontracts/context.py:95-116  (fields, abridged)
self.__caller_id = caller_id     # Address or ContractId that made the call
self.__vertex    = vertex_data   # data about the calling transaction
self.__block     = block_data    # data about the block executing it
self.__actions   = actions       # the NCActions, grouped by token
```

Note what is not there: no live transaction object, no storage handle, no clock. The Context exposes `caller_id`, the actions, and read-only views of the vertex and block — and nothing else. It is a `@final` class with private slots, and the Runner even passes the method a copy (`runner.py:675`, `ctx.copy()`) so that even a malicious contract cannot mutate the original and confuse the engine's later checks. The actions are grouped by token (`__group_actions__`, `context.py:49`) so a method can ask "what is being deposited in HTR?" with `ctx.get_single_action(token_uid)`.

View methods, by contrast, receive **no** Context (`types.py:294`, `validate_has_not_ctx_arg`). A view is a pure read — it has no caller to speak of and moves no tokens — so handing it a Context would be meaningless. That asymmetry is the cleanest way to tell the two method kinds apart: public takes a Context and may write; view takes none and may only read.

39.A.5 The decorators: `@public`, `@view`, `@fallback`

The decorators in `types.py` are how a Blueprint author marks intent, and how the framework knows which methods are callable and how. They do almost no work at call time; their job is to stamp a marker attribute onto the function so the Runner can check it later.

```
# hathor/nanocontracts/types.py:206-209
def _set_method_type(fn: Callable, method_type: NCMethodType) -> None:
    if hasattr(fn, NC_METHOD_TYPE_ATTR):
        raise BlueprintSyntaxError('method must be annotated with at most one method type')
    setattr(fn, NC_METHOD_TYPE_ATTR, method_type)   # stamp PUBLIC / VIEW / FALLBACK
```

`@public` (`types.py:248`) stamps `NCMethodType.PUBLIC` and validates the method's shape at decoration time — before the node ever runs: it must have `self`, it must have a `ctx: Context` argument (`validate_has_ctx_arg`, `:270`), and its argument types must be

serializable. It also records which actions the method allows. `@view` (`types.py:284`) stamps `VIEW` and validates the opposite: it must **not** take a `ctx` (`validate_has_not_ctx_arg`, `:294`). `@fallback` (`types.py:312`) marks a single catch-all method, called `fallback`, that the Runner invokes when a transaction names a method the Blueprint does not have — Hathor's equivalent of a default handler.

Doing this validation at decoration time, rather than at call time, means a malformed Blueprint fails to load rather than failing mysteriously deep inside a transaction. That matters because on-chain Blueprint code is run through these same decorators when it is loaded from the ledger (§39.B.5); a Blueprint that does not obey the rules is rejected before it can ever be instantiated.

39.A.6 The Runner and the syscall surface

We now have a Blueprint (a class), a contract (an instance with state in the trie), and a transaction with a nano-header that names a method to call. The **Runner** is the engine that puts them together: it takes "call method `m` on contract `c` with these arguments and actions," fetches the right Blueprint code, instantiates it, runs the method under resource limits, and either commits the resulting state changes or throws them away.

The Runner is the heart of the subsystem — `hathor/nanocontracts/runner/runner.py`, 1467 lines. We will not read all of it; we will trace the one path that matters and name the guard-rails along the way.

The public entry point used by consensus is `execute_from_tx` (`runner.py:163`). It reads the nano-header, performs the replay check, and dispatches to either "create a contract" or "call a method":

```
# hathor/nanocontracts/runner/runner.py:163-194  (heavily abridged)
def execute_from_tx(self, tx: Transaction) -> None:
    nano_header = tx.get_nano_header()
    contract_id = (ContractId(VertexId(tx.hash))          # creating: id is the tx hash
                   if nano_header.is_creating_a_new_contract()
                   else ContractId(VertexId(nano_header.nc_id)))   # calling: id is nc_id

    # replay protection: the caller's seqnum must advance, by a small step
    current = self.block_storage.get_address_seqnum(Address(nano_header.nc_address))
    diff = nano_header.nc_seqnum - current
    if diff <= 0 or diff > MAX_SEQNUM_JUMP_SIZE:
        raise NCFail(f'invalid seqnum (diff={diff})')

    context = nano_header.get_context()
    nc_args = NCRawArgs(nano_header.nc_args_bytes)
    if nano_header.is_creating_a_new_contract():
        self.create_contract_with_nc_args(contract_id, BlueprintId(...), context, nc_args)
    else:
        self.call_public_method_with_nc_args(contract_id, nano_header.nc_method, context, nc_args
```

The **seqnum**[10] check deserves a word. Each caller address has a monotonically increasing sequence number stored in the block trie. A new call must use a higher seqnum than the last one seen (`get_address_seqnum`, `block_storage.py:164`), by no more than `MAX_SEQNUM_JUMP_SIZE` (10). This stops an attacker from replaying a signed transaction twice and stops calls from being reordered arbitrarily, while the small allowed jump leaves room for a few calls in flight.

Dispatching the call

Follow the "call a method" branch into `_execute_public_method_call` (`runner.py:606`), the method every public call funnels through, whether it came from a transaction or from another contract:

```
# hathor/nanocontracts/runner/runner.py:623-647  (abridged)
self._validate_context(ctx)
changes_tracker = self._create_changes_tracker(contract_id)        # staging area
blueprint = self._create_blueprint_instance(blueprint_id, changes_tracker)
method = getattr(blueprint, method_name, None)

if method is None:                          # no such method → try the fallback
    fallback_method = getattr(blueprint, NC_FALLBACK_METHOD, None)
    if fallback_method is None:
        raise NCMethodNotFound(...)
    method = fallback_method
    ...
else:
    if not is_nc_public_method(method):  # refuse to call a @view as if public
        raise NCInvalidMethodCall(...)
    parser = Method.from_callable(method)            # capture the signature
    args = self._validate_nc_args_for_method(parser, nc_args)   # decode + type-check args
```

Three things are worth naming:

1. **The method must be `@public`.** A transaction cannot reach a `@view` method through this path; calling a non-public method is an error (`runner.py:644`). The decorator marker from §39.A.5 is what makes this check possible.
2. **Arguments are decoded and re-encoded** (`_validate_nc_args_for_method`, `runner.py:690`). The raw bytes from the header are deserialized against the method's declared types and a fresh copy of each argument is built. This both type-checks the input and severs any shared references, so one contract cannot smuggle a mutable object into another and mutate it behind its back.
3. **The actual run is metered.** The method is not called directly. It is handed to a `MeteredExecutor`:

```
# hathor/nanocontracts/runner/runner.py:675
ret = self._metered_executor.call(method, args=(ctx.copy(), *args))
```

That single line is where contract code finally executes — under the resource bound we discuss next. After it returns, the call's balance changes are validated to be non-negative (`runner.py:678`) and the staged changes may be committed.

The syscall surface: `BlueprintEnvironment`

Recall that a Blueprint instance holds exactly one thing, its `__env`, exposed as `self.syscall`. That environment is the **only** door from contract code back into the node, and it is a deliberate application of the **proxy pattern**[11] (Chapter 3): the contract never touches storage or the Runner directly; it talks to a `BlueprintEnvironment`, which forwards each request to the Runner under controlled conditions.

```python
# hathor/nanocontracts/blueprint_env.py:35-58  (abridged)
@final
class BlueprintEnvironment:
    """A class that holds all possible interactions a blueprint may have with the system."""
    __slots__ = ('__runner', '__log__', '__storage__', '__cache__')

    @property
    def rng(self) -> NanoRNG:
        return self.__runner.syscall_get_rng()      # deterministic randomness

    def get_contract_id(self) -> ContractId:
        return self.__runner.get_current_contract_id()
```

Through `self.syscall`, a contract can read its own balance, get a deterministic random number, and — the consequential one — **call another contract**. Inter-contract calls go through the Runner's `syscall_call_another_contract_public_method` (`runner.py:327`), which enforces the rules that make composition safe: a contract cannot call itself (`runner.py:342`), recursion is capped at `MAX_RECURSION_DEPTH = 100` (`runner.py:120`), the total number of calls in one transaction is capped at `MAX_CALL_COUNTER = 250` (`runner.py:121`), and re-entrancy[12] is forbidden unless the method explicitly opted into it. There is also a proxy (delegatecall-style) variant, `syscall_proxy_call_public_method` (`runner.py:391`), where another Blueprint's code runs against the calling contract's storage — the same primitive Ethereum calls `DELEGATECALL`, used for upgradeable libraries.

The reason all of this is funnelled through one environment object is the **isolation** constraint from §39.A.1. If a contract could reach `tx_storage` or the reactor directly, the sandbox would be meaningless. By giving it a single, hand-built `BlueprintEnvironment` whose every method is a deliberate, checked syscall, the node decides exactly what a contract is permitted to do — and nothing leaks through.

39.A.7 Metered execution: why, and how

We have reached the single most consequential safety mechanism, and one where the code holds a surprise that you should know about.

Why bound execution at all

Recall constraint two from §39.A.1: a contract must not be able to run forever or exhaust memory. This is not a hypothetical worry; it is a direct consequence of a deep result in computer science. The **halting problem**[13] says there is no general algorithm that can look at an arbitrary program and decide whether it will eventually stop or loop forever. So the node cannot inspect a Blueprint and prove it terminates. A contract author could write `while True: pass`, by accident or by malice.

If the node ran that code to completion, it would never finish — and because every node runs every contract, a single such transaction would freeze the entire network at once. This is the contract-world version of the **"never block the reactor"** rule from Chapter 2: the node's event loop must keep turning, and any single task must be bounded.

Since you cannot decide termination in advance, the standard answer — Ethereum's gas, every smart-contract platform's answer — is to meter execution: give each call a finite budget, charge the budget down as the code runs, and **abort** when it hits zero. You do not need to know whether the program halts; you only need to guarantee that your execution of it halts. The budget is called **fuel**[14] (Hathor's word; Ethereum says gas). The same logic applies to memory: cap it, and abort if the cap is exceeded.

How Hathor sets it up

The budget lives in `MeteredExecutor` (`hathor/nanocontracts/metered_exec.py`). The Runner creates one per call, seeded from settings:

```
# hathor/nanocontracts/runner/runner.py:294
self._metered_executor = MeteredExecutor(
    fuel=self._initial_fuel,        # NC_INITIAL_FUEL_TO_CALL_METHOD
    memory_limit=self._memory_limit,  # NC_MEMORY_LIMIT_TO_CALL_METHOD
)
```

The intended cost model is one fuel unit per Python bytecode opcode executed — there is a per-opcode cost table, `FUEL_COST_MAP = [1] * 256` (`metered_exec.py:33`), and the comment beside it points at Python's `sys.settrace` hook, the standard way to run a callback on every line/opcode. Two exception types stand ready: `OutOfFuelError` and `OutOfMemoryError` (`metered_exec.py:36`, `:40`).

The actual run compiles the call into a tiny code stub and `exec`s it inside a restricted environment:

```python
# hathor/nanocontracts/metered_exec.py:80-111  (abridged)
def call(self, func, /, *, args):
    from hathor import NCFail
    from hathor.nanocontracts.custom_builtins import EXEC_BUILTINS
    env = {'__builtins__': EXEC_BUILTINS, '__func__': func, '__args__': args, '__result__': None}
    code = compile('__result__ = __func__(*__args__)', '<blueprint>', 'exec', ...)
    try:
        exec(code, env)
    except NCFail:
        raise
    except Exception as e:
        raise NCFail from e          # ANY error becomes a clean NC failure
    return env['__result__']
```

Two parts of this are real and load-bearing today:

- **The sandbox is real.** `EXEC_BUILTINS` (from `custom_builtins.py`) is a hand-curated replacement for Python's builtins. Dangerous functions — `eval`, `exec`, `open`, `__import__` and many more — are removed or replaced with versions that raise `NCDisabledBuiltinError` (`custom_builtins.py:262`). `import` is reduced to an allow-list, and even `range`, `all`, `any`, `enumerate`, `filter` are re-implemented in pure Python (`custom_builtins.py:91` onward) so that they execute opcode by opcode in the interpreter rather than dropping into uncountable C code. That last detail is a tell: re-implementing builtins in Python is precisely what you do so that an opcode-counting meter can see them.
- **Failures are contained.** Any exception a contract raises is converted to `NCFail` (`metered_exec.py:109`). The Runner never lets a contract's error escape as an arbitrary Python exception; it always becomes a clean, well-typed failure that consensus knows how to handle (§39.B.6).

The honest caveat (a correction to the surface story)

Here is the surprise, and this book reports the code over the tidy narrative. In this branch, `MeteredExecutor` **stores** the `fuel` and `memory_limit` but the `exec` / `call` methods shown above **do not actually decrement fuel or enforce the memory cap** — there is no `sys.settrace` callback installed, and the `# TODO: cost for each opcode` comment at `metered_exec.py:32` confirms the per-opcode accounting is unfinished. The `OutOfFuelError` / `OutOfMemoryError` types are defined and are caught by the on-chain-blueprint loader (`on_chain_blueprint.py:201-206`), but nothing on the normal call path currently raises them.

What this means precisely: the **architecture** for metering is in place — the budget is threaded through, the exception types exist, the builtins are deliberately written to be meterable — but the **enforcement hook itself is scaffolded, not yet wired**, in the version on this branch. The sandbox (which functions a contract may call) is fully active; the resource bound (how long it may run) is not yet enforced at this layer. Treat

the metering design as the intended and documented model, and the live enforcement as work in progress. If you are reading a later revision, check whether `call` now installs a trace function — that is where the meter will live.

39.A.8 Worked trace: deploy and call a Counter

Pull Part A together with one concrete story. We use the `Counter` Blueprint from §39.A.3. The example is realistically shaped — it uses the real API surface verified above — though the exact Counter is illustrative; `hathor-core` ships no built-in counter (the built-in registry `_blueprints_mapper` is empty by default, `blueprints/__init__.py:21`), so in practice this code would arrive as an on-chain Blueprint (§39.B.5).

Step 0 — the Blueprint exists. Either it is registered in the catalog, or — far more commonly — its source has been published in an `OnChainBlueprint` transaction whose hash is the blueprint id.

Step 1 — create a contract. Alice broadcasts a transaction whose nano-header has `nc_method = "initialize"`, `nc_id = <blueprint id>`, no actions, and her signature. When a block confirms it, consensus hands the tx to the Runner's `execute_from_tx`. Because the method is `initialize`, `is_creating_a_new_contract()` is true, so the Runner takes the **create** branch:

```
contract_id  = ContractId(tx.hash)        # the new contract's identity = this tx's hash
blueprint_id = BlueprintId(nc_id)         # which template to instantiate
create_contract_with_nc_args(contract_id, blueprint_id, ctx, nc_args)   # runner.py:928
```

`create_contract_with_nc_args` checks the contract does not already exist, records its blueprint id in storage, then runs `initialize` through the same `_execute_public_method_call` path as any other call. Inside, `self.count = 0` fires the `count` field's descriptor, which writes `0` into a fresh slice of the trie. After `initialize` returns, the Runner verifies every declared field was initialized (`_check_all_field_initialized`, `runner.py:314`) — a contract may not leave a field unset — and commits. The contract now exists, with `count = 0` and an empty balance.

Step 2 — call `increment`. Bob broadcasts a transaction with `nc_method = "increment"`, `nc_id = <contract id from step 1>`, a higher seqnum than his last call, and his signature. When confirmed, the Runner takes the **call** branch:

```
1.  seqnum check: Bob's nc_seqnum advances by 1..10          → ok
2.  changes_tracker = staging layer over the contract's storage
3.  blueprint = Counter(env)            # instance with env pointing at the tracker
4.  method = blueprint.increment        # found; it is @public            (runner.py:626)
5.  args   = decode+typecheck nc_args_bytes (none here)                  (runner.py:647)
6.  metered_executor.call(increment, args=(ctx.copy(),))                 (runner.py:675)
        → self.count += 1   reads 0 from trie, writes 1 to the tracker
7.  balances validated non-negative; staged change committed             (runner.py:307)
```

The change is written first to a `NCChangesTracker` — a staging layer — and only flushed to the real trie once the call succeeds. We see why that staging matters in the next step.

Step 3 — a call fails. Suppose `increment` instead did `self.count += 1` and then hit an error (a bad assertion, a forbidden builtin, an explicit `raise NCFail`). The `MeteredExecutor.call` catches it and re-raises `NCFail`. Because the write went to the tracker, not the trie, the increment is discarded — the contract's `count` stays at its previous value. **A failed contract call has no effect on state.** And, as Part B shows, the failing transaction is itself voided. This atomicity — all of a call's writes land, or none do — is the same guarantee a database transaction gives you, and it is what makes contracts safe to compose.

That is the whole runtime in miniature: a transaction names a method, the Runner fetches the Blueprint, runs the method under a (future) fuel bound inside a real sandbox, stages the writes, and either commits them or throws them away. Part B is where those committed writes actually go.

39.B — The State

Part A ended with `self.count = 1` being "committed to the trie." Part B is that trie: what it is, why it is shaped the way it is, and how a contract's typed attributes turn into bytes inside it. Then we close the loop back to consensus.

39.B.1 Why a trie, and what a trie is

A contract's state is just key→value data: `count → 1`, `balances[HTR] → 500`, and so on. You could store that in a plain dictionary on disk. The reason Hathor does not — the reason it uses a **trie** with a particular hashing scheme — comes back to constraint four from §39.A.1: **verifiable state.** Every node must be able to agree, with a single short fingerprint, that its copy of all contract state is identical to everyone else's. A plain dictionary gives you the data but no cheap, tamper-evident fingerprint of the whole of it. A Merkle structure does.

Build the idea in two layers.

Layer one: a trie (prefix tree)

A **trie**[15] (also "prefix tree" or, in its compressed form, "radix trie") stores keys by spelling them out along a path from the root. Instead of one big table, you have a tree where each step down consumes part of the key. Keys that share a prefix share the top of their path:

```
keys: "cat", "car", "dog"

        (root)
        /     \
     "ca"     "dog" → value(dog)
     /   \
   "t"   "r"
    |     |
  val    val
 (cat)  (car)
```

The two useful properties: lookups follow the key character by character (fast), and — crucially for us — **the structure is fully determined by its contents**. Two tries holding the same set of key→value pairs are the same tree, regardless of what order the pairs were inserted. That order-independence is what lets two nodes that processed the same contract writes end up with byte-identical state.

Layer two: Merkle hashing → a state root

Now make every node carry a hash of itself and all its descendants. Each node's id is `hash(its own data + the ids of its children)`. Change any value anywhere in the tree, and that node's id changes, which changes its parent's id, all the way up — so the **root's id changes**. This is a **Merkle tree**[16], and the root id is a one-hash fingerprint of the entire dataset. This single value is the **state root**[17].

Two nodes can now compare all of their contract state by comparing one 32-byte number. If the roots match, the states are provably identical; if they differ, something somewhere is different. That is the verifiability we needed.

A **Patricia trie**[18] is the practical, compressed version: chains of single-child nodes are collapsed into one node holding the whole shared substring, so the tree stays shallow. Hathor's combination — a Patricia (compressed) trie with Merkle (hash-linked) node ids — is the data structure at `hathor/nanocontracts/storage/patricia_trie.py`.

39.B.2 The trie in code

The implementation is faithful to the picture above and small enough to read in one sitting. A node:

```python
# hathor/nanocontracts/storage/patricia_trie.py:40-65  (abridged)
@dataclass(kw_only=True, slots=True)
class Node:
    key: bytes
    length: int
    content: Optional[bytes] = None          # the stored value, if this key was set
    children: DictChildren = ...
    _id: Optional[NodeId] = None             # this node's Merkle id
```

The Merkle id is computed exactly as described — hash of key, content, and sorted child ids:

```python
# hathor/nanocontracts/storage/patricia_trie.py:67-79
def calculate_id(self) -> NodeId:
    h = hashlib.sha256()
    h.update(self.key)
    if self.content is not None:
        h.update(self.content)
    for child_id in sorted(self.children.values()):   # SORTED → order-independent
        h.update(child_id)
    return NodeId(h.digest())
```

`sorted(...)` is the line that guarantees order-independence: children are hashed in id order, never in insertion order, so the same set of children always yields the same parent id.

The class makes one design choice explicit in its docstring: **the nodes are immutable.**

```
# hathor/nanocontracts/storage/patricia_trie.py:94-100
class PatriciaTrie:
    """... All nodes are immutable. So every update will create a new path of nodes
    from leaves to a new root. ..."""
```

An update never edits a node in place. It copies the node it touches and rebuilds a fresh path of copies from that node up to a new root (`_update` → `_build_path`, `patricia_trie.py:281`, `:237`). The old nodes are untouched and still reachable from the old root. This is **persistence**[19] in the data-structure sense: every past root id still names a complete, valid snapshot of the state at that moment. That property is exactly what lets consensus re-execute or roll back contract state during a reorg (§39.B.6) — it can pick up an earlier root and rebuild from there, because that earlier state was never overwritten.

Reading and writing go through `get` / `update`, and committing flushes the staged new nodes to the backing store:

```
# hathor/nanocontracts/storage/patricia_trie.py:373-381, 118
def update(self, key, content):  # writes a value; may change self.root
def get(self, key, *, root_id=None):  # reads a value (optionally at a past root)
def commit(self):  # flush local node changes to the database
```

`root.id` is the state root. Hold that thought — it is the value that travels up into consensus.

39.B.3 Two levels of trie: contract state and block state

A single trie per contract is not the whole story, because the node must fingerprint all contracts at once. Hathor uses **two levels** of trie.

The contract trie holds one contract's state. It is wrapped by `NCContractStorage` (`hathor/nanocontracts/storage/contract_storage.py:137`), which gives the raw `key→bytes` trie a typed face: attributes, balances, and metadata, each under a distinct tag so they cannot collide:

```
# hathor/nanocontracts/storage/contract_storage.py:40-58  (abridged)
class _Tag(Enum):
    ATTR     = b'\0'      # a state attribute like `count`
    BALANCE  = b'\1'      # the contract's balance of a token
    METADATA = b'\2'      # e.g. which blueprint this contract uses

class AttrKey(TrieKey):
    def __bytes__(self) -> bytes:
        return _Tag.ATTR.value + hashlib.sha256(self.key).digest()
```

So `self.count` is stored under the trie key `b'\0' + sha256(b'count')`, its balance of HTR under `b'\1' + <htr-token-id>`, and its blueprint id under a metadata key. The contract's state root is `NCContractStorage.get_root_id()` (`contract_storage.py:384`) — its trie's root id.

The block trie sits one level up. `NCBlockStorage` (`hathor/nanocontracts/storage/block_storage.py:59`) holds a trie that maps each contract id to that contract's state root, plus per-token descriptions and the per-address seqnums:

```python
# hathor/nanocontracts/storage/block_storage.py:82-89
def get_contract_root_id(self, contract_id):       # contract → its state root
    return self._block_trie.get(bytes(ContractKey(contract_id)))

def update_contract_trie(self, nc_id, root_id):    # record a contract's new root
    self._block_trie.update(bytes(ContractKey(nc_id)), root_id)
```

Because the block trie is itself a Merkle trie over the contracts' roots, its own root — `NCBlockStorage.get_root_id()` (`block_storage.py:95`) — is a single fingerprint of **the entire nano-contract state of the whole network** at that block. This is the value the node stores in the block's metadata as `nc_block_root_id`, and it is how two nodes confirm they agree on all contract state with one comparison. Picture it:

```
       block trie root  =  nc_block_root_id  (in the block's metadata)
                │
   ┌────────────┼──────────────────┐
   ▼            ▼                  ▼
contract_A  contract_B  …  contract_Z      (block trie maps id → state root)
state root  state root     state root
   │
   ▼   (each is itself the root of that contract's own trie)
 count → 1
 balance[HTR] → 500
 blueprint_id → …
```

Fetching a contract's storage walks down both levels: `get_contract_storage` looks up the contract's root in the block trie, then opens a `PatriciaTrie` at that root and wraps it in an `NCContractStorage` (`block_storage.py:112`).

39.B.4 Typed fields: how `self.count` becomes bytes

We left a thread dangling in Part A: the metaclass turns `count: int` into a `Field` descriptor. Now we can see where it leads.

A `Field` is a Python **descriptor** — an object that defines `__get__` / `__set__` / `__delete__` and so intercepts attribute access on the class that holds it:

```python
# hathor/nanocontracts/fields/field.py:76-85  (abridged)
class Field(Generic[T]):
    def __set__(self, instance: Blueprint, value: T) -> None:
        node = self._container_node_factory.build(instance)
        node.set_value(self._prefix, value)        # serialize + write to the trie

    def __get__(self, instance: Blueprint, owner=None) -> T:
        node = self._container_node_factory.build(instance)
        return node.get_value(self._prefix)        # read from the trie + deserialize
```

So `self.count += 1` is really read `count` from the trie, add one, write it back to the trie — the contract author's `+= 1` hides a load and a store. The `_prefix` is the field name (`b'count'`), which becomes part of the trie key. The factory distinction between a scalar field and a container field (a `dict`, `list`, `set`, `deque` attribute) is what lets `self.balances['alice'] = 5` write only the one entry's key rather than re-serializing the whole map — containers under `hathor/nanocontracts/fields/` map each element to its own trie key.

But "write `value` to the trie" needs one more piece: the trie stores **bytes**, and `value` is a Python `int`, `str`, dataclass, whatever. The translation is the job of an `NCType` — a serialization codec keyed by the declared type:

```python
# hathor/nanocontracts/nc_types/nc_type.py:28-34  (abridged)
class NCType(ABC, Generic[T]):
    """models a type with a known type signature and how it will be (de)serialized.
    Used for NC method-call args, and for the values stored in NC properties."""
```

There is one `NCType` subclass per supported value shape — `BoolNCType`, `StrNCType`, `BytesNCType`, sized-int types, `OptionalNCType`, collection types, `make_dataclass_nc_type` for dataclasses, and more, all under `hathor/nanocontracts/nc_types/`. When the metaclass builds the `Field` for `count: int`, it resolves `int` to the right `NCType`, and that codec is what turns `1` into the bytes stored under `b'\0' + sha256(b'count')`. The same `NCType` machinery serializes a method's arguments into `nc_args_bytes` for the nano-header (§39.A.4) and deserializes them back when the Runner decodes the call — one type-to-bytes system, used at both the storage boundary and the call boundary.

This typed-storage layer is why a Blueprint declares its state with ordinary annotations and gets persistence, serialization, and a verifiable hash with no extra ceremony. The author writes `count: int`; the framework supplies a descriptor, a codec, a trie key, and a Merkle root.

Staging: the changes tracker

One detail from Part A's failure case (§39.A.8, step 3) lives here. The contract a method writes to is not the live `NCContractStorage` directly — it is an `NCChangesTracker` (`hathor/nanocontracts/storage/changes_tracker.py`) layered over it. The tracker subclasses `NCContractStorage` but buffers every write — attribute changes, balance diffs, authority changes — in memory:

```
# hathor/nanocontracts/storage/changes_tracker.py  (shape)
class NCChangesTracker(NCContractStorage):
    # buffers attribute writes, balance diffs, authority diffs;
    # .commit() applies them to the underlying storage; .block() discards them.
```

If the call succeeds, the tracker is committed and its buffered writes flow into the real trie. If the call raises `NCFail`, the tracker is discarded and the writes evaporate — the atomicity guarantee from §39.A.8. The tracker is also what lets the Runner compute a call's net balance change and check conservation (`_validate_balances`, `runner.py:519`) before anything is made permanent.

39.B.5 On-chain Blueprints: code stored on the ledger

Where does a Blueprint's source code come from? Two places.

A **built-in Blueprint** is one shipped with the node and registered in the catalog. The catalog maps a blueprint id to a Python class:

```
# hathor/nanocontracts/catalog.py:29-33
class NCBlueprintCatalog:
    def get_blueprint_class(self, blueprint_id) -> Type['Blueprint'] | None:
        return self.blueprints.get(blueprint_id, None)
```

The built-in registry `_blueprints_mapper` (`blueprints/__init__.py:21`) is **empty by default** on this branch — built-ins are populated from settings (`generate_catalog_from_settings`, `catalog.py:37`), so in practice almost every Blueprint is the second kind.

An **on-chain Blueprint** is one whose source code lives on the ledger, published as a transaction. This is the `OnChainBlueprint` vertex you were promised back in Chapter 25. It is an ordinary `Transaction` subclass (version `ON_CHAIN_BLUEPRINT`) that carries a compressed blob of Python source:

```
# hathor/nanocontracts/on_chain_blueprint.py:155-185  (abridged)
class OnChainBlueprint(Transaction):
    """On-chain blueprint vertex to be placed on the DAG of transactions."""
    def __init__(self, ..., code: Optional[Code] = None, ...):
        if not self._settings.ENABLE_NANO_CONTRACTS:
            raise RuntimeError('NanoContracts are disabled')
        self.nc_pubkey: bytes = b''        # author's public key
        self.nc_signature: bytes = b''     # author's signature
        self.code = code or Code(CodeKind.PYTHON_ZLIB, b'', self._settings)
```

The `Code` object holds the source `zlib`-compressed (`Code.from_python_code`, `on_chain_blueprint.py:128`), with size caps to stop someone publishing a multi-megabyte blob. The **blueprint id is the transaction's own hash** (`blueprint_id`, `:189`) — publishing code mints its identity, exactly as creating a contract mints the contract's id from its tx hash.

When the node first needs to run an on-chain Blueprint, it must turn that stored source text into a runnable class. This is where the source is `exec`-ed — under its own metered budget, separate from the per-call budget:

```
# hathor/nanocontracts/on_chain_blueprint.py:193-208  (abridged)
def _load_blueprint_code_exec(self):
    fuel  = self._settings.NC_INITIAL_FUEL_TO_LOAD_BLUEPRINT_MODULE
    memory_limit = self._settings.NC_MEMORY_LIMIT_TO_LOAD_BLUEPRINT_MODULE
    metered_executor = MeteredExecutor(fuel=fuel, memory_limit=memory_limit)
    try:
        env = metered_executor.exec(self.code.text)        # run the module's top level
    except OutOfFuelError as e:
        raise OCBOutOfFuelDuringLoading from e
    except OutOfMemoryError as e:
        raise OCBOutOfMemoryDuringLoading from e
    blueprint_class = env[BLUEPRINT_EXPORT_NAME]            # the @export-ed class
    return blueprint_class, env
```

Two safeguards stand out. First, running the module is metered too — loading code is itself a contract-like activity that an attacker could weaponize (a module whose top level loops forever), so it gets the same fuel/memory treatment (with the same caveat from §39.A.7 about enforcement being scaffolded — note the loader is one of the few places that actually catches the fuel/memory exceptions). Second, the loaded module must export exactly one Blueprint via the `@export` decorator (`types.py:299`), which stashes the class under a well-known name `__blueprint__`; the loader pulls that class out (`on_chain_blueprint.py:207`). The result is cached on the vertex (`_load_blueprint_code`, `:210`) so the source is parsed and executed at most once per node lifetime.

The sandbox from §39.A.7 is what makes running arbitrary on-chain source even thinkable: the `exec` happens with `EXEC_BUILTINS`, so the published module cannot open files, import freely, or call `eval`. A Blueprint is source code anyone can publish; the jail is the only thing standing between that and the node's machine.

39.B.6 Execution meets consensus

We can now close the largest loop in the book: how a contract call actually runs in the life of the node, and how a failure is folded back into the consensus you learned in Chapter 32.

The key fact, and it surprises people: **nano-contracts do not execute when their transaction is first received.** A nano-contract transaction sits in the mempool, verified but not run, until a **block confirms it**. Execution happens at block consensus time, for all the NC transactions a block confirms, together. The reason is determinism and ordering: two conflicting calls must be run in an order every node agrees on, and only a block gives a canonical ordering of the transactions beneath it.

> **Recap — voiding and `voided_by` (full treatment Ch. 10 & 32).** Consensus never deletes a vertex; it voids one by adding a marker to its `voided_by` set in metadata. A voided vertex stays in storage but is excluded from the canonical ledger. In Chapter 32 you saw two markers: a tx's own hash (it lost a conflict) and `SOFT_VOIDED_ID`. Nano-contracts add a third — and this section is where the forward-reference from Chapter 32 is paid off. → full treatment of consensus in Ch. 32.

Sorting the calls

When a block is processed, the NC transactions it confirms are first put into a **deterministic order** by the sorter (`hathor/nanocontracts/sorter/random_sorter.py`). It is not a plain timestamp sort; it is a random topological sort, seeded by the block's hash:

```
# hathor/nanocontracts/sorter/random_sorter.py:29-40  (abridged)
def random_nc_calls_sorter(block, nc_calls):
    sorter = NCBlockSorter.create_from_block(block, nc_calls)
    seed = hashlib.sha256(block.hash).digest()           # determinism from block hash
    order = sorter.generate_random_topological_order(seed)
    ...
```

Two properties are designed in. **Topological**: if call B depends on call A (A is an input of B, or they share a caller seqnum), A runs first — Kahn's algorithm over the dependency DAG (`NCBlockSorter`, `random_sorter.py:57`). **Random but deterministic**: among calls with no dependency between them, the order is shuffled by a pseudo-random generator seeded from the block hash, so it is unpredictable to an attacker yet identical on every node. The randomness denies anyone the ability to game execution order; the determinism keeps consensus intact.

Running the block

`NCBlockExecutor.execute_block` (`hathor/nanocontracts/execution/block_executor.py:131`) walks the sorted calls and runs each through the Runner. It is written as a pure generator: it `yield`s an effect for each step rather than mutating anything itself, so the part that decides "did this succeed or fail" is cleanly separated from the part that applies the consequences. Each call produces one of three results:

```
# hathor/nanocontracts/execution/block_executor.py:38-60  (abridged)
NCTxExecutionSuccess   # ran cleanly
NCTxExecutionFailure   # raised NCFail
NCTxExecutionSkipped   # was already voided (e.g. by an earlier failing call)
```

The per-call RNG seed is derived from the block hash, the tx hash, and the current state root (`block_executor.py:180-186`) — deterministic, but unique per call, so two contracts in the same block get different randomness.

Applying the effects, and voiding failures

The companion `NCConsensusBlockExecutor` (`hathor/nanocontracts/execution/consensus_block_executor.py`) consumes those effects and applies them inside the consensus context. The success and failure branches are where the whole chapter lands:

```
# hathor/nanocontracts/execution/consensus_block_executor.py:234-298  (heavily abridged)
case NCTxExecutionSuccess(tx=tx, runner=runner):
    tx_meta.nc_execution = NCExecutionState.SUCCESS
    runner.commit()                         # flush the staged state into the trie
    tx.storage.indexes.non_critical_handle_contract_execution(tx)   # update indexes
    context.nc_exec_success.append(tx)    # queue a pubsub event (Ch 30)

case NCTxExecutionFailure(tx=tx, runner=runner, exception=exception, traceback=tb):
    on_failure(tx)                          # void the tx with NC_EXECUTION_FAIL_ID
    self._nc_log_storage.save_logs(tx, runner.get_last_call_info(), (exception, tb))
```

On **success**: the Runner's staged changes are committed to the contract's trie, the indexes are updated, and a pubsub event fires so downstream listeners learn the contract executed. On **failure**: the transaction is voided via the `on_failure` callback, which adds the marker `NC_EXECUTION_FAIL_ID` (the literal bytes `b'nc-fail'`, `nanocontracts/__init__.py:25`) to the tx's `voided_by`. The executor's own invariant check spells out the resulting metadata exactly:

```
# hathor/nanocontracts/execution/consensus_block_executor.py:202-208  (abridged)
case NCExecutionState.SUCCESS:  assert tx_meta.voided_by is None
case NCExecutionState.FAILURE:  assert tx_meta.voided_by == {tx.hash, NC_EXECUTION_FAIL_ID}
case NCExecutionState.SKIPPED:  assert NC_EXECUTION_FAIL_ID not in tx_meta.voided_by
```

This is the third voiding marker promised by Chapter 32. A transaction can be perfectly valid — correct signature, sufficient weight, no double-spend — and still be voided, because the contract method it called raised `NCFail`. Validity (Chapter 31) asks "is this transaction well-formed?"; nano-execution asks "did the program it invokes succeed?" Both must hold for the transaction to count, and a failure in the second is recorded distinctly so anyone inspecting the ledger can see why the tx was voided: not a conflict, not a structural error, but a contract that rejected the call.

At the end of the block, the executor commits the block storage and records the block-trie root in the block's metadata:

```
# hathor/nanocontracts/execution/consensus_block_executor.py:313-319  (abridged)
case NCEndBlock(block=block, block_storage=block_storage, final_root_id=final_root_id):
    block_storage.commit()
    meta = block.get_metadata()
    meta.nc_block_root_id = final_root_id     # the verifiable fingerprint of ALL contract state
    context.save(block)
```

`nc_block_root_id` is the §39.B.3 block-trie root — one hash that fingerprints every contract's state after this block. Because the trie is persistent (§39.B.2), a **reorg**[20] is handled by `execute_chain` (`consensus_block_executor.py:101`) resetting the affected blocks' `nc_block_root_id` to `None` and re-executing from the common ancestor's root — the older states were never overwritten, so they are still there to rebuild from.

39.B.7 How it all plugs into the lifecycle

Step back and place nano-contracts on the node's spine — the life-of-a-node story from Chapter 0 — for a single contract-calling transaction:

```
1.  A tx with a NanoHeader arrives over the network          (P2P / ingestion, Ch 33–35)
2.  VERIFICATION checks the nano-header is well-formed        (Ch 31; gated: ENABLE_NANO_CONT
3.  The tx enters the mempool — NOT yet executed
4.  A BLOCK confirms it
5.  CONSENSUS sorts the block's NC calls deterministically    (random_sorter, §39.B.6)
6.  The RUNNER executes each call under a metered budget       (runner.py, §39.A.6–7)
       ├─ success → state committed to the contract trie       (§39.B.2–4)
       └─ failure → tx voided with NC_EXECUTION_FAIL_ID         (Ch 32, §39.B.6)
7.  The block-trie root (nc_block_root_id) is saved into block metadata  (§39.B.3)
8.  Indexes updated; pubsub events fired                        (Ch 28, Ch 30)
9.  Whole subsystem is OFF unless the NANO_CONTRACTS feature is active   (Ch 38)
```

Every box has been a chapter. Nano-contracts do not replace any of them; they thread through all of them. A nano-contract transaction is still a vertex (Chapter 25), still serialized the bespoke way (Chapter 26), still stored in RocksDB (Chapter 27), still verified (Chapter 31), still ordered by consensus (Chapter 32), still ingested through the vertex handler (Chapter 33). The nano-contracts package adds exactly one new thing to that pipeline: at consensus time, when a block confirms a call, run the program and commit or void. That is the whole subsystem's contribution, and now you can defend why each of its eighty files exists to make that one step deterministic, bounded, isolated, and verifiable.

Recap

Concept	What it is	Where in code
Blueprint	the contract template (a Python class)	`blueprint.py:126` ; metaclass `:35`
Contract	a live instance, with state + balance	id = creating tx hash (`runner.py:169`)

Concept	What it is	Where in code
`@public` / `@view` / `@fallback`	method markers; public writes + takes `ctx`, view reads only	`types.py:248`, `:284`, `:312`
State attribute	a type annotation → a trie-backed `Field` descriptor	`blueprint.py:79`; `fields/field.py:76`
NanoHeader	the on-tx instruction: which contract, method, args, actions	`transaction/headers/nano_header.py:100`
NCAction	deposit/withdrawal/grant/acquire across the contract boundary	`types.py:359-491`
Context	immutable "who/what" passed to a public method	`context.py:35`
Runner	dispatches a call to a Blueprint method	`runner/runner.py:117`; `execute_from_tx:163`
MeteredExecutor	fuel + memory bound (enforcement scaffolded)	`metered_exec.py:44`
Sandbox	restricted builtins; disabled `eval` / `open` / `import`	`custom_builtins.py`
Patricia/Merkle trie	order-independent, hash-linked state store	`storage/patricia_trie.py:94`
NCContractStorage	one contract's state (attrs/balances/metadata)	`storage/contract_storage.py:137`
NCBlockStorage	maps contract id → state root; its root = all NC state	`storage/block_storage.py:59`
State root	one hash fingerprinting state; per-block `nc_block_root_id`	`contract_storage.py:384`; `block_storage.py:95`
Changes tracker	stages writes so a failed call rolls back	`storage/changes_tracker.py`
OnChainBlueprint	a tx carrying compressed Python Blueprint source	`on_chain_blueprint.py:155`

Concept	What it is	Where in code
Execution ⇄ consensus	run at block confirm; failure → `NC_EXECUTION_FAIL_ID`	`execution/consensus_block_executor.py`
`NC_EXECUTION_FAIL_ID`	voiding marker `b'nc-fail'` for a failed contract call	`nanocontracts/__init__.py:25`
Feature gate	the subsystem is off unless `NANO_CONTRACTS` is active	`feature_activation/feature.py:33`

Nano-contracts are the node's second world: a small, sandboxed Python runtime stacked on the ledger you spent the book learning. A Blueprint is a class; a contract is an instance whose attributes live in a Merkle trie; a transaction's nano-header names a method to call; the Runner runs that method under a (designed) resource bound inside a real sandbox; and consensus — not the moment of arrival — is when the program actually executes, committing verifiable state on success and voiding the transaction with `NC_EXECUTION_FAIL_ID` on failure. The two design choices to carry forward are Python Blueprints over a bespoke VM (familiar language, careful jail) and a per-contract, per-block Merkle root (verifiable state in one hash). Note the live caveat: the metering enforcement hook is scaffolded but not yet wired on this branch — the budget is threaded through and the sandbox is active, but fuel is not yet decremented per opcode. The next chapter turns from contracts that hold money to the keys that own it: **Chapter 40 — Wallets and crypto**, where addresses, signatures, and the `nc_script` that authorizes every call you saw here finally get their full treatment.

---

1. A smart contract is a program stored on a ledger whose code is executed and agreed upon by every node in the network, so it can hold and move tokens according to its own public rules with no trusted operator. Hathor's variant is called a nano-contract. ↩
2. Deterministic execution means the same inputs always produce exactly the same outputs on every machine. For consensus this is mandatory: if two nodes computed different results for the same contract call, they would disagree on the ledger. It rules out wall-clock time, true randomness, and reading external data. ↩
3. Bytecode is a low-level, compact instruction format that a program is compiled into and that a virtual machine executes, rather than running the original source directly. Ethereum stores contract bytecode on-chain; Hathor stores Python source and runs it on the interpreter. ↩
4. The EVM (Ethereum Virtual Machine) is Ethereum's purpose-built virtual machine: a stack-based interpreter with its own instruction set that every Ethereum node runs to execute contract bytecode. Hathor has no equivalent separate machine — contracts run on the node's own (sandboxed) Python interpreter. ↩
5. Gas is Ethereum's unit of computational cost: each operation costs a fixed amount of gas, the transaction carries a gas budget, and execution aborts if the budget runs out. It is the mechanism that makes unbounded loops harmless. Hathor's equivalent budget is called fuel. ↩

6. A Blueprint is Hathor's term for a contract template — a Python class defining the contract's state and methods. One Blueprint can back many independent contracts, just as one class can have many object instances. ↩
7. A metaclass is a class whose instances are themselves classes. Defining `class Counter(Blueprint)` runs the metaclass's code, which can inspect and rewrite the class before it exists. Hathor uses this to convert a Blueprint's type annotations into storage descriptors. ↩
8. A descriptor is a Python object that customizes what happens when an attribute is read, written, or deleted, by defining `__get__` / `__set__` / `__delete__`. Properties are descriptors. Hathor's `Field` is a descriptor that routes `self.count` to the trie instead of to memory. ↩
9. A header here is a self-describing block of extra bytes appended to a transaction's serialized form, carrying optional structured data. The nano-header carries the instructions for a contract call; a separate fee header carries fee data. ↩
10. A sequence number (seqnum) is a per-caller counter that must strictly increase with each call. It prevents replay: an attacker cannot resubmit a previously signed transaction, because its seqnum is no longer higher than the last one the node accepted. ↩
11. The proxy pattern (Chapter 3) places a stand-in object between a client and a real service so the stand-in can control, restrict, or mediate access. The `BlueprintEnvironment` is a proxy: contract code talks to it, and it forwards each request to the Runner under controlled, checked conditions. ↩
12. Re-entrancy is when a contract, mid-call, calls back into itself (often via another contract) before the first call finished. It is a classic source of exploits because the contract's state may be half-updated. Hathor forbids it by default and a method must explicitly opt in with `allow_reentrancy=True`. ↩
13. The halting problem is the proven impossibility of writing a general algorithm that decides, for any program and input, whether it will eventually stop or run forever. Its consequence here: the node cannot prove a contract terminates, so it must instead bound execution with a finite budget. ↩
14. Fuel is Hathor's name for the metered execution budget — the equivalent of Ethereum's gas. The design charges fuel per executed bytecode operation and aborts when it reaches zero; this branch threads the budget through but does not yet decrement it (see §39.A.7). ↩
15. A trie (or prefix tree) stores keys by spelling them out along a path from the root, so keys with a shared prefix share the top of their path. A radix/Patricia trie is the compressed form that collapses single-child chains into one node. ↩
16. A Merkle tree is a tree in which every node's identifier is a hash of its own data plus its children's identifiers. Any change anywhere alters the root hash, so the single root value is a tamper-evident fingerprint of the entire dataset. ↩
17. A state root is the Merkle root of a state store — one hash that uniquely fingerprints all of the state. Two parties can confirm their states are identical by comparing this one value. Hathor keeps a per-contract state root and a per-block root over all contracts. ↩
18. A Patricia trie is a compressed (radix) trie: chains of single-child nodes are merged into one node holding the whole shared substring, keeping the tree shallow and lookups efficient. Hathor combines this with Merkle hashing for verifiable state. ↩
19. A persistent data structure (in the data-structure sense, unrelated to disk persistence) is one where updates produce a new version while leaving every previous version intact and accessible. Hathor's trie copies a path on each write, so every past state root still names a complete snapshot — which is what makes reorgs cheap to undo. ↩
20. A reorg (reorganization) happens when consensus switches the canonical chain to a different branch, so some previously-canonical blocks become voided and others take their place. For nano-contracts this means re-executing the affected blocks from an earlier state root. ↩